
SUPERRELoRA: MODIFIED RELoRA METHOD USING LOW-RANK UPDATES

Timofey Glukhikh, Ivan Oseledets, Anh-Huy Phan
Skolkovo Institute of Science and Technology

ABSTRACT

Large language models are becoming increasingly larger, with the number of parameters growing correspondingly. Training or adapting them with full-rank updates is expensive, while a single low-rank adapter has limited expressive power. We propose SuperReLoRa, a multi-cycle extension of ReLoRa that periodically merges LoRa adapters into the base weights and reinitializes them. Unlike ReLoRa, which relies on random reinitialization, SuperReLoRa enforces column-space orthogonality of successive adapters at each reinit, so that new cycles start in input subspaces disjoint from previous ones and the accumulated update can increase in rank. We compare SuperReLoRa against LoRa and ReLoRa under a matched training budget.

1 INTRODUCTION

Large networks with hundreds of billions of parameters are extremely difficult to retrain, yet we often need to adapt a model to a new task or continue pretraining under a limited compute budget.

Aghajanyan et al. (2020) show that pretrained language models have a low “intrinsic dimension”. The Lottery Ticket Hypothesis Frankle & Carbin (2019) similarly suggests that overparameterization can be reduced. Low-rank approximation of large matrices and tensors has a long history in numerical linear algebra (see, e.g., Lubich & Oseledets (2013), Sulz et al. (2023)).

Inspired by this, Hu et al. (2021) hypothesized that weight updates during adaptation also have low intrinsic rank and proposed LoRa. ReLoRa Lialin et al. (2023) extends LoRa by periodically merging low-rank adapters into the base weights and reinitializing them, so that the total update can become higher-rank over many cycles. However, ReLoRa does not explicitly prevent successive adapters from occupying overlapping subspaces. We propose SuperReLoRa, which keeps the multi-cycle merge-and-reinit structure but enforces column-space orthogonality of new LoRa factors at each reinitialization. Empirically, we compare LoRa, ReLoRa, and SuperReLoRa under identical hyperparameters and compute.

2 RELATED WORK

Parameter-efficient fine-tuning. Parameter-efficient fine-tuning (PEFT) reduces the cost of adapting large pretrained models by updating only a small subset of parameters. Early adapter methods introduced compact trainable modules for transfer learning in vision Rebuffi et al. (2017; 2018) and NLP Houlsby et al. (2019); Pfeiffer et al. (2021). Alternative PEFT families include bias-only tuning Ben-Zaken et al. (2022), prompt- and prefix-tuning Lester et al. (2021); Li & Liang (2021), and other lightweight reparameterizations He et al. (2022). Our work follows the low-rank adapter line rather than sequential modules or prompt-based approaches.

Low-rank adapters. LoRa Hu et al. (2021) freezes pretrained weights W and learns a low-rank update $\delta W = sW_A W_B$. Subsequent variants improve allocation or efficiency of this factorization: AdaLoRa Zhang et al. (2023) adapts rank via SVD-based parameterization, DoRA Liu et al. (2024) decomposes updates into magnitude and direction, VeRA Kopiczko et al. (2024) shares frozen random projections across layers, and QLoRa Dettmers et al. (2023) combines LoRa with quantized base weights. Practical implementations further reduce memory and compute Cherniuk et al. (2023);

Dettmers et al. (2023). These methods typically train a *single* low-rank adapter of fixed rank r , and therefore do not explicitly accumulate a higher-rank total update over the course of training.

High-rank training through low-rank updates. ReLoRA Lialin et al. (2023) observes that repeated merge-and-reinitialize of LoRA adapters can approximate a higher-rank update $\Delta W = \sum_i s W_A^i W_B^i$ while keeping the optimizer state small. After each merge into W , ReLoRA hard-resets the adapter matrices and partially resets Adam moments Kingma & Ba (2017), because retained moments otherwise continue to optimize the previous subspace; it also uses a jagged learning-rate schedule with warm restarts. Random reinitialization encourages, but does not guarantee, that successive adapters explore new directions: nothing prevents a new cycle from recovering the same column space as an earlier one.

Our positioning. Prior PEFT methods either keep a fixed low-rank adapter Hu et al. (2021); Zhang et al. (2023); Liu et al. (2024); Kopiczko et al. (2024); Dettmers et al. (2023) or periodically merge adapters and hard-reset them to accumulate higher-rank updates Lialin et al. (2023). SuperReLoRA follows the multi-cycle merge-and-reinit template of ReLoRA, but additionally enforces *column-space orthogonality* of the new W_A against an accumulated orthonormal basis of past adapters at each reinit. The goal is that successive LoRA cycles start in disjoint input subspaces, so that the summed update ΔW can increase in effective rank with the number of cycles. In experiments we compare LoRA, ReLoRA, and SuperReLoRA under a matched budget.

3 OUR METHOD

We build on ReLoRA Lialin et al. (2023) and train with successive LoRA cycles. We freeze pre-trained weights $W \in \mathbb{R}^{m \times n}$ and parameterize each cycle update as a low-rank product

$$\begin{aligned} \delta W &= s W_A W_B, \\ W_A &\in \mathbb{R}^{in \times r}, \quad W_B \in \mathbb{R}^{r \times out}, \end{aligned} \tag{1}$$

where $s \in \mathbb{R}$ is a fixed scaling factor (typically α_{LoRA}/r).

Every q steps we merge the current adapter into the base weights and start a new LoRA cycle. After N cycles the total update is

$$\Delta W = \sum_{i=1}^N \delta W_i = \sum_{i=1}^N s W_A^{(i)} W_B^{(i)}. \tag{2}$$

If successive factors reuse the same input subspace, the sum in equation 2 need not increase rank. ReLoRA relies on random reinitialization and optimizer pruning to encourage new directions, but does not enforce independence of subspaces.

Column-space orthogonality at reinit. Our modification is to force each new cycle to start in an input subspace orthogonal to previous ones. Let U be an orthonormal basis of the column space spanned by all previously merged adapters $\{W_A^{(i)}\}_{i < k}$. At the beginning of cycle k we reinitialize $W_A^{(k)}, W_B^{(k)}$ as in ReLoRA and then project

$$W_A^{(k)} \leftarrow (I - UU^\top) W_A^{(k)}. \tag{3}$$

Thus $W_A^{(k)\top} U = 0$ at cycle start: the new adapter uses input directions disjoint from past cycles. We update U after each merge by appending an orthonormal basis of the just-merged W_A (e.g., via QR). Orthogonality is enforced *only at reinit*; during the cycle the factors may drift under gradient updates. This yields a multi-cycle LoRA scheme whose successive updates are initialized to be column-space orthogonal, aiming for $\text{rank}(\Delta W)$ to grow with the number of cycles.

As in ReLoRA, Adam moments Kingma & Ba (2017) associated with the old subspace would otherwise pull the new factors back into previous directions. We therefore prune optimizer states at merge-and-reinit and restart the learning rate with a jagged warmup schedule. The full procedure is given in Algorithm 1.

Algorithm 1 SuperReLoRa. θ denotes model parameters; $\hat{\theta}$ the model after replacing linear layers by LoRA adapters; M, V are Adam states; η follows a jagged schedule; q is the merge/reinit period; U stores an orthonormal basis of past adapter column spaces.

Require: θ, M, V, η, q

```

1:  $U \leftarrow []$  {empty basis}
2: for  $t$  in warm-start steps do
3:   Update  $\theta, M, V, \eta$  {standard training}
4: end for
5: for layer in model layers do
6:   if layer is linear then
7:     Replace layer by LoRA( $W, W_A, W_B$ ); freeze  $W$ 
8:   end if
9: end for
10: for  $t$  in training steps do
11:   Update  $\hat{\theta}, M, V$  {LoRA training step}
12:   if  $\text{MOD}(t, q) = 0$  then
13:     for  $l$  in linear layers do
14:        $W \leftarrow W + s W_A W_B$  {full merge}
15:       Append orthonormal basis of  $\text{col}(W_A)$  to  $U$ ; re-orthonormalize  $U$ 
16:        $W_A \leftarrow \text{kaiming\_init}(W_A)$ ;  $W_B \leftarrow 0$  {reinit}
17:        $W_A \leftarrow (I - U U^\top) W_A$  {column-space orthogonalization}
18:        $M_{W_A}, V_{W_A}, M_{W_B}, V_{W_B} \leftarrow \text{prune}(\cdot)$ 
19:     end for
20:     Restart  $\eta$  warmup
21:   end if
22: end for
23: return  $\theta$ 

```

4 EXPERIMENTS

Goal. We evaluate whether enforcing column-space orthogonality at merge-and-reinit improves over standard LoRA and over ReLoRA when the training budget is matched. The only algorithmic difference between ReLoRA and SuperReLoRa is the orthogonal projection of W_A at reinitialization equation 3; LoRA uses the same adapter parameterization but never merges.

Setup. All methods fine-tune `nicholasKluge/TeenyTinyLlama-160m` on WikiText-2 Merity et al. (2017) (train split for training, validation for evaluation). Unless stated otherwise we use rank $r=8$, LoRA scaling $\alpha_{\text{LoRA}}=16$, learning rate 2×10^{-4} , batch size 8, sequence length 512, and three epochs. Adapters are attached to attention and MLP projections (`q/k/v/o_proj`, `gate/up/down_proj`). For ReLoRA and SuperReLoRa we merge every $q=500$ steps, prune 99% of low-magnitude Adam moments, and apply a short jagged learning-rate warmup after each merge. LoRA uses the same optimizer and schedule without merge-and-reinit. Experiments run on the Zhores HPC cluster (SLURM partition `gpu_devel`, single GPU).

Methods.

- **LoRA:** frozen base weights, single low-rank adapter, no merge.
- **ReLoRA:** periodic full merge into W , random reinit of W_A, W_B , Adam prune, jagged LR warmup.
- **SuperReLoRa:** same as ReLoRA, plus column-space orthogonalization of W_A against the accumulated basis U at each reinit.

Metrics. We report training loss, validation perplexity, and token-level next-token accuracy. To probe the intended mechanism we also track (i) the residual overlap $\|W_A^\top U\|_F$ immediately after reinit and (ii) the dimension of the accumulated basis U (or an effective-rank estimate of the summed update ΔW) as a function of the number of cycles.

Table 1: Matched-budget comparison on WikiText-2 (TeenyTinyLlama-160M). Lower perplexity is better. Numbers to be filled after training.

Method	Train loss ↓	Val. PPL ↓	Acc. ↑
LoRA	–	–	–
ReLoRA	–	–	–
SuperReLoRa	–	–	–

Results. Table 1 summarizes the main comparison.

We expect SuperReLoRa to match or improve upon ReLoRA on validation perplexity while exhibiting lower post-reinit subspace overlap and a faster growth of effective rank across cycles; LoRA provides a fixed-rank reference without multi-cycle accumulation.

5 CONCLUSIONS

We presented SuperReLoRa, a multi-cycle LoRA method that combines ReLoRA-style merge-and-reinit with enforced column-space orthogonality of successive adapters at reinitialization. Under a matched training budget we compare LoRA, ReLoRA, and SuperReLoRa on language-model adaptation. The orthogonal reinit aims to start each cycle in an input subspace disjoint from previous ones, so that the accumulated update ΔW can increase in effective rank. Future work includes stronger orthogonality constraints during the cycle, scaling to larger models, and broader downstream evaluations.

REFERENCES

- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning, 2020.
- Eyal Ben-Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language models. *arXiv preprint arXiv:2106.10199*, 2022.
- Dmytro Cherniuk, Alexander Mikhalev, and Ivan Oseledets. Run lora run: Faster and lighter lora implementations. *arXiv preprint arXiv:2312.03415*, 2023.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.
- Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019.
- Junxian He, Xiaochang Zhu, Ling Liu, and Wei Ma. Exploring efficient tuning methods for pre-trained language models. *arXiv preprint arXiv:2206.03350*, 2022.
- Neil Houlsby, Alex Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 2790–2799, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. Vera: Vector-based random matrix adaptation, 2024.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.

-
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pp. 4582–4597, 2021.
- Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. Stack more layers differently: High-rank training through low-rank updates, 2023.
- Shih-Yang Liu, Chia-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation, 2024.
- Christian Lubich and Ivan Oseledets. A projector-splitting integrator for dynamical low-rank approximation, 2013.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- Jonas Pfeiffer, Andreas Rücklé, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. *arXiv preprint arXiv:2005.00247*, 2021.
- Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *Advances in Neural Information Processing Systems*, 30, 2017.
- Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Efficient parametrization of multi-domain deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 8119–8127, 2018.
- Dominik Sulz, Christian Lubich, Gianluca Ceruti, Igor Lesanovsky, and Federico Carollo. Numerical simulations of long-range open quantum many-body dynamics with tree tensor networks, 2023.
- Tianyi Zhang, Minglei Li, Jingqing Yang, and Tong Zhang. Adalora: Adaptive low-rank adaptation of large language models. *arXiv preprint arXiv:2301.03485*, 2023.